



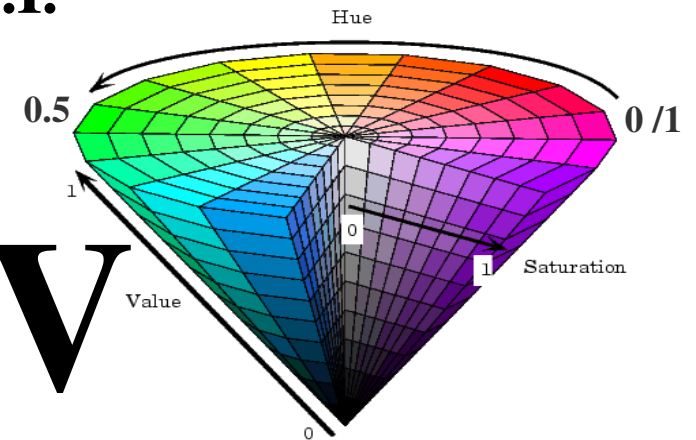
COLOR

Software Engineering and Data Science

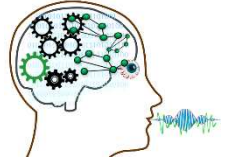
SEIS 766: Vision A.I.

Dr. Chih Lai

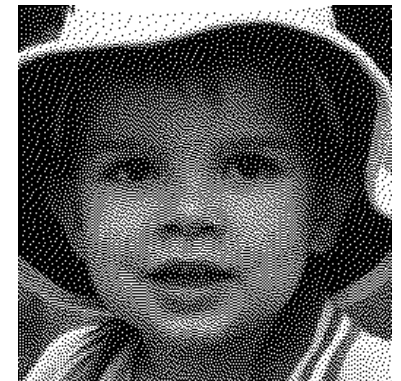
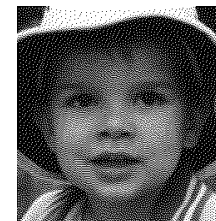
HSV



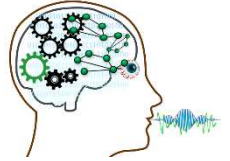
Outline



- Color and Color Space
- **RGB** Color Space, Gray-Scale, Black/White
 - Impact of image resizing to colors
- Indexed Colors and Usage
 - Color Motion Map
- ➔ ■ Problems / Pitfalls in Dealing with Colors
 - **Dithering** / **Image Resizing** / **Saving Images** **Augmentation**
- HSV Color Space and Usage
 - Adjust Light Condition, Histogram Equalization



Problem 1: Dithering



- Convert gray image into “black / white” with or without **dithering**
- Which one to choose???

Input
Gray
Image



B/W **with** dithering



B/W **with** dithering



WHY to Black & White???

Visualization

Object ID foreground vs background

For quick training on smaller CNNs

Document OCR recognition

Morphological Operations



threshold > **var** as white **without** dithering



0.4039

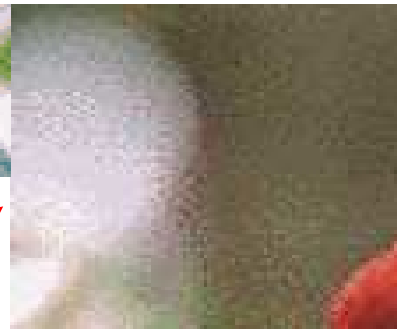
0.3

0.1

Threshold to white >

Indexed Color Reduction

- **Dithering** Impact, with & without
- Which one to choose?? WHY??



32-color Dithering

Parrots_Index_Dither.fig

32-color No Dithering



```
Parrots = imread('.\Images\Parrots.jpg');
[a, amap] = rgb2ind(Parrots, 32);
[b, bmap] = rgb2ind(Parrots, 32, 'nodither');
figure, subplot(1,2,1),imshow(a, amap);
subplot(1,2,2), imshow(b, bmap)
```

from PIL import Image

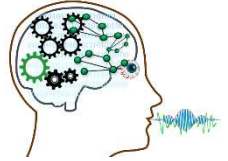
image.convert('P', palette=Image.ADAPTIVE, colors=32, dither=Image.NONE)

VAI_Slide_IdxCOLOR.ipynb

% ← pay attention here

VAI_Slide_IdxCOLOR.m

Medical Image– Dithering vs. No Dithering



Input



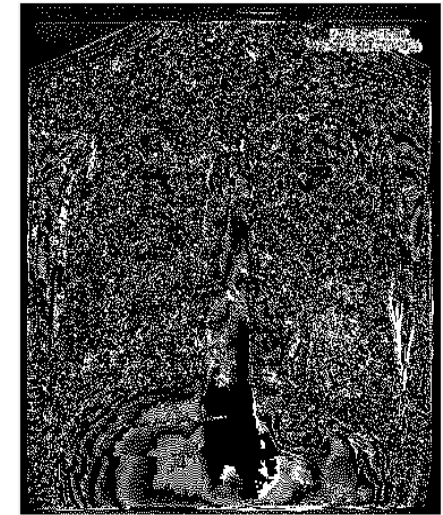
Dithering (a)



No Dithering (b)



$\Delta = a - b$



Lung_Compare.fig

Lung_Diff.fig

```
lung = imread('.\Images\lung.jpg');
```

```
lung = imresize(lung, 0.5);
```

```
[a, amap]=rgb2ind(lung, 32);
```

```
[b, bmap] = rgb2ind(lung, 32, 'nodither');
```

```
figure, subplot(1,2,1),imshow(a, amap); subplot(1,2,2), imshow(b, bmap)
```

```
c = imabsdiff(a, b);
```

```
figure, imshow(c * 80) # scale up to see the differences
```

```
from PIL import Image, ImageChops
```

```
image.convert('P', palette=Image.ADAPTIVE, colors=32, dither=Image.NONE)
```

```
diff = ImageChops.difference(a, b)
```

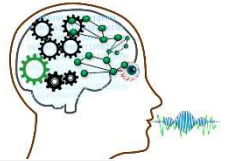
VAI_Slide_IdxCOLOR.ipynb

```

RGB = imread('peppers.png');
figure,
subplot(2,2,1), imshow(RGB); title('0');
subplot(2,2,2), imshow(imresize(RGB, 0.8)); title('0.8');
subplot(2,2,3), imshow(imresize(RGB, 0.3)); title('0.3');
subplot(2,2,4), imshow(imresize(RGB, 0.3, 'nearest')); title('0.3 nearest')

```

Problem 2: Image Resizing



choose this one??

0.3

`imresize(image, 0.3)`

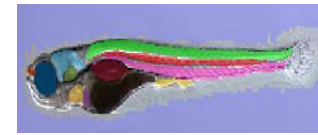
`cv2.resize(image, (width, height))`

choose this one??

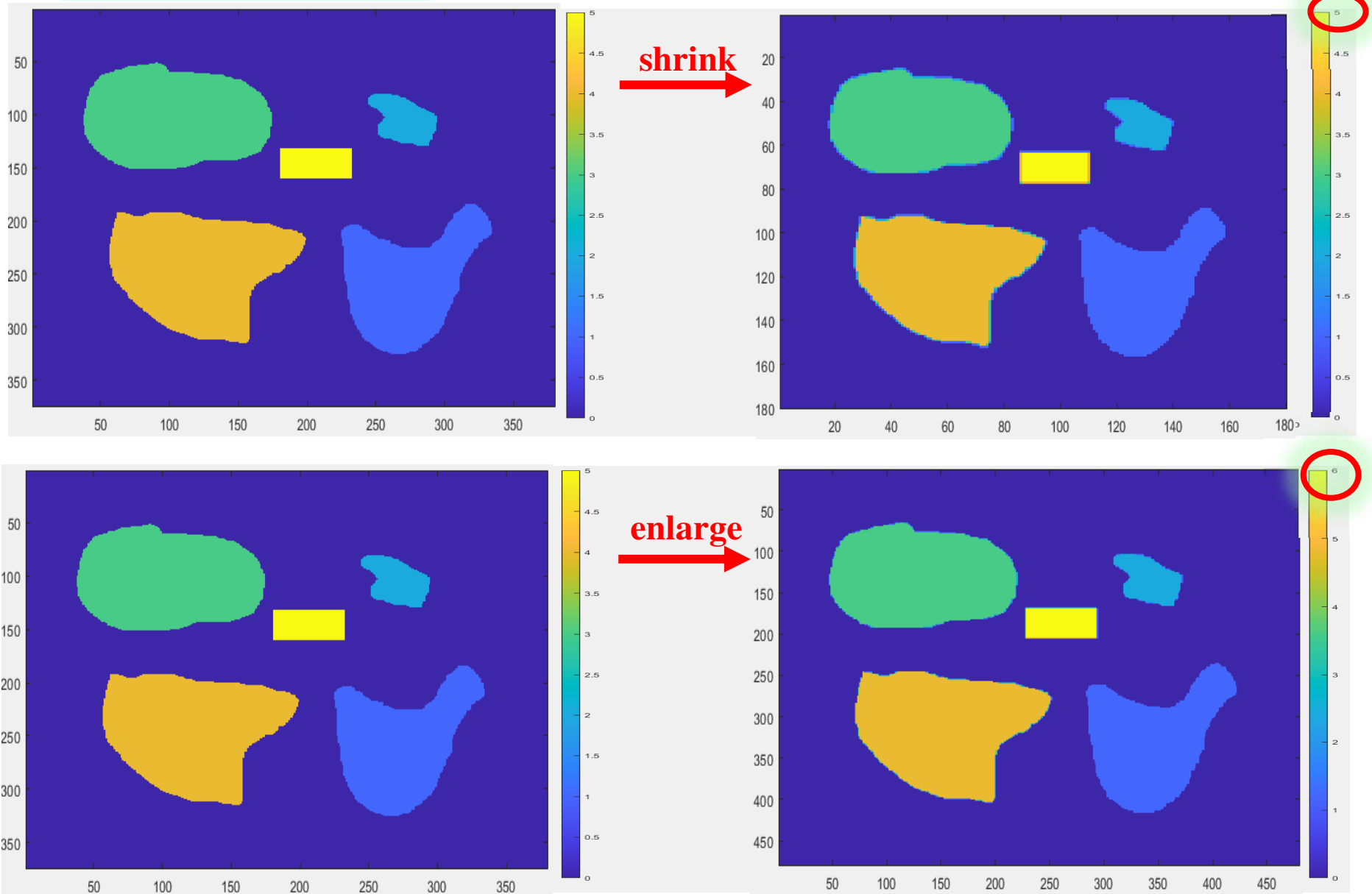
0.3 near



Resizing Images



VAI_Slide_ShrinkEnlarge.m



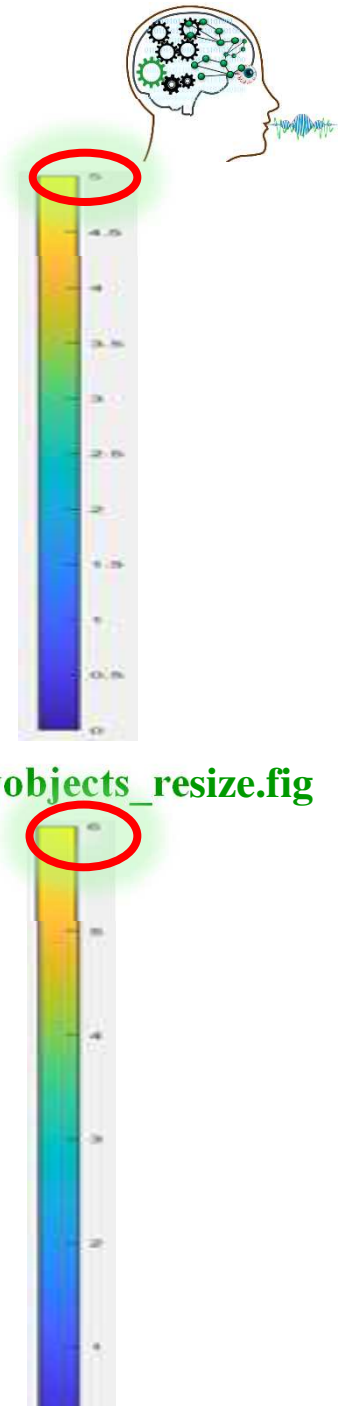
toyobjects_resize.fig

Zoom in to Check Jagging Resizing Result



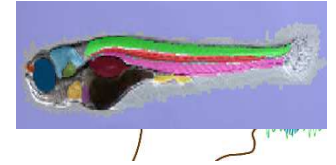
`unique(Img(:))`

`np.unique(Img)`

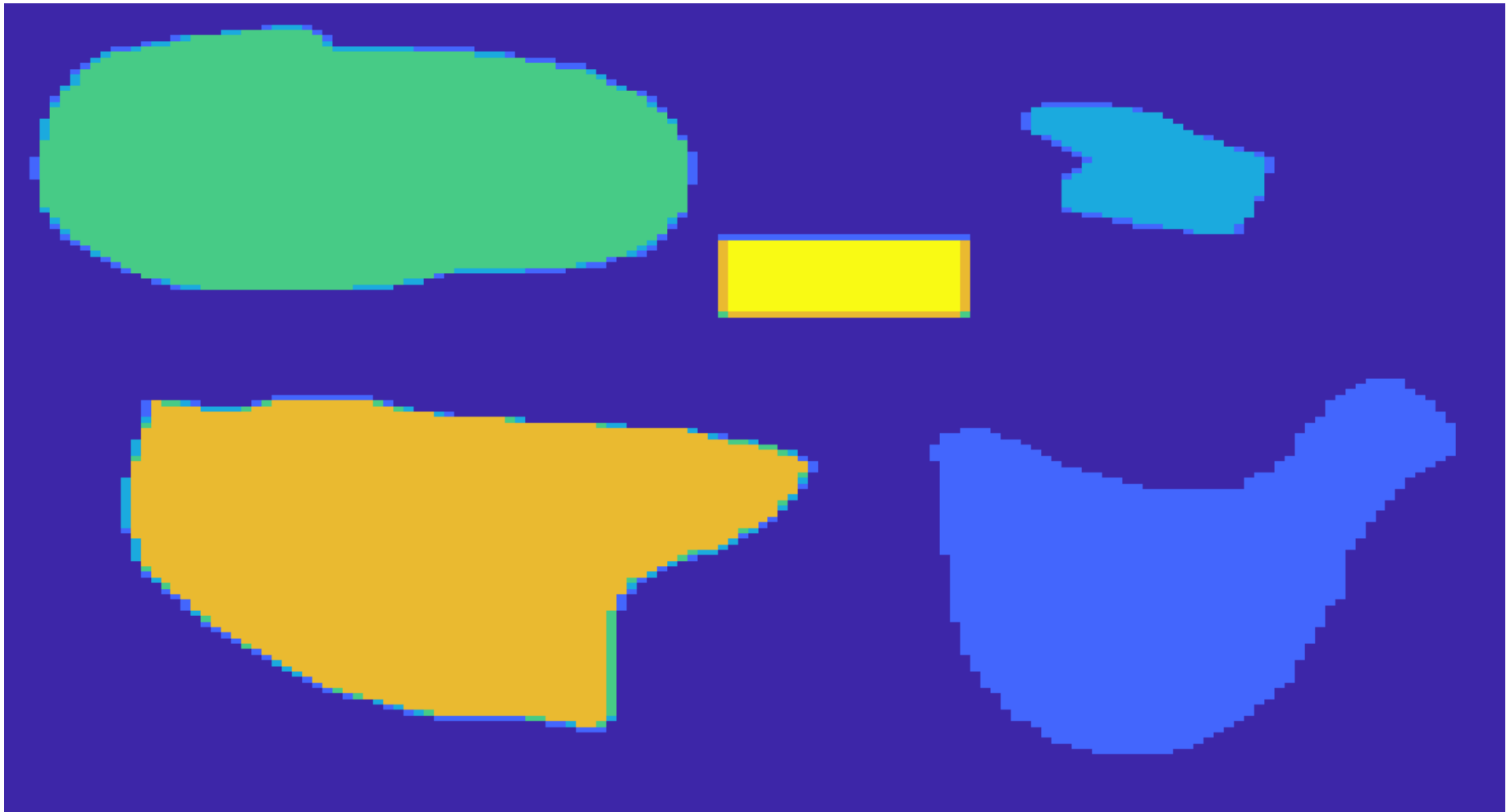




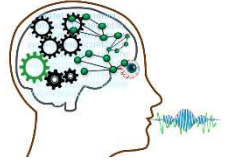
After Shrinking 50% (Zoom -in View)



- Either, jagging / aliasing effect
- Or, output contains **additional colors**



View Input Gray-Scale Images as RGB, HOW??



■ How do we view a gray-scale image as RGB?

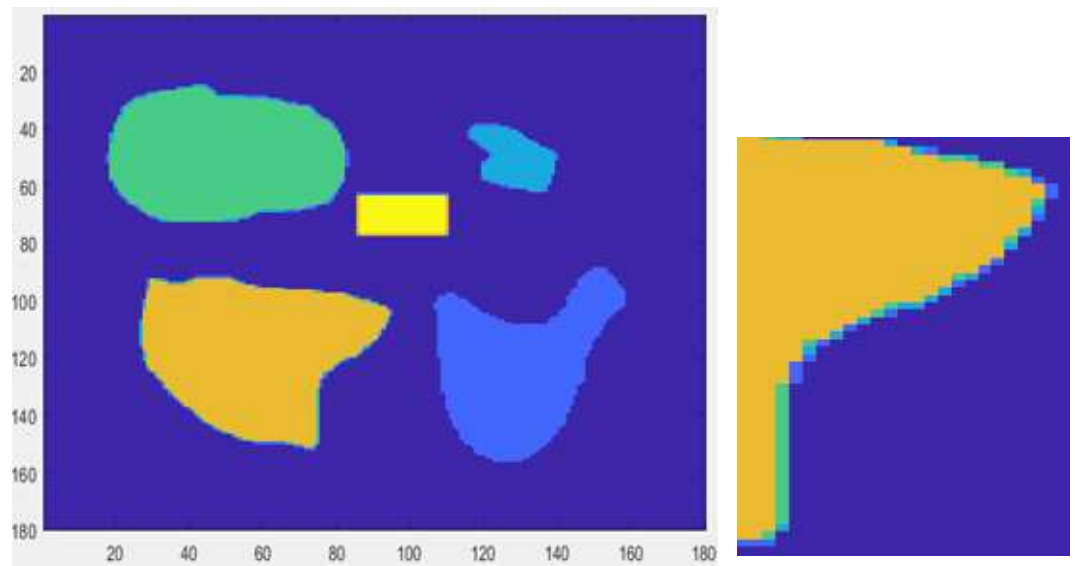
- **Python** → `plt.imshow(I, cmap = 'jet')` **import** matplotlib.pyplot as plt
- **MATLAB** → `imagesc(I)`



Folder: \Images

toyobjects.png

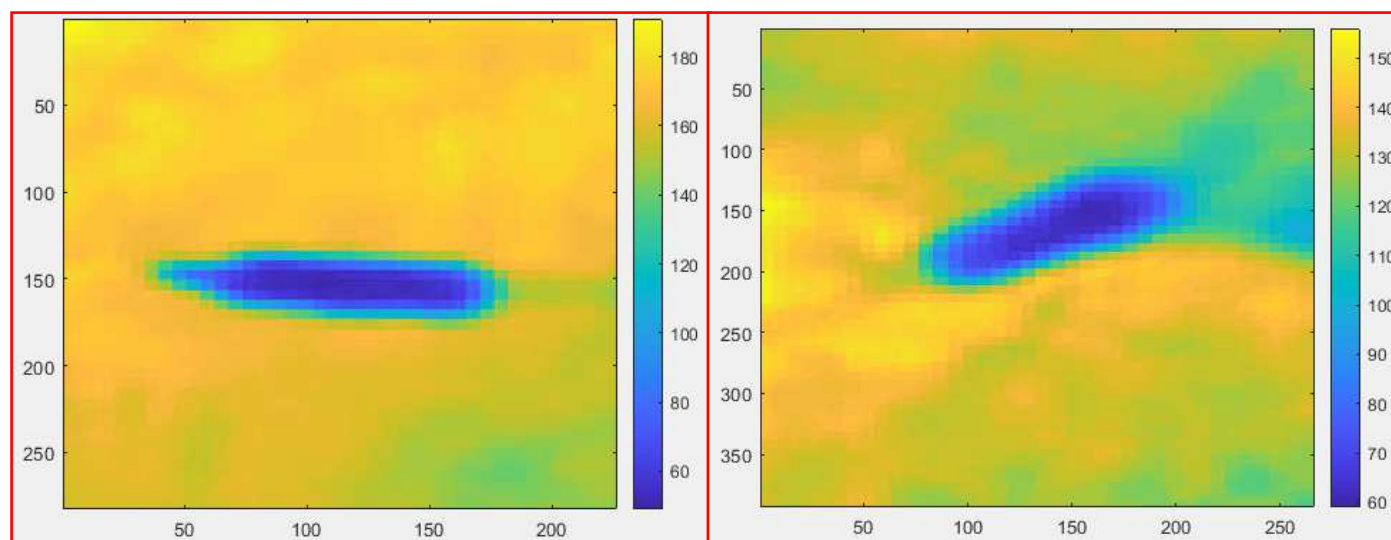
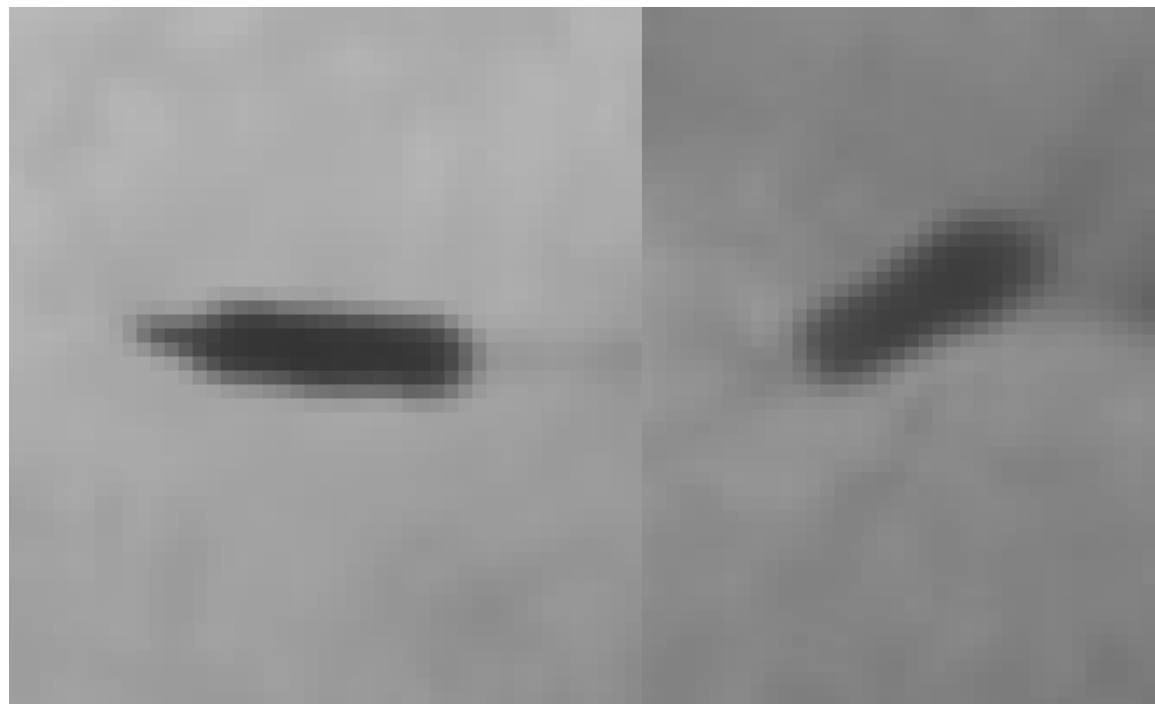
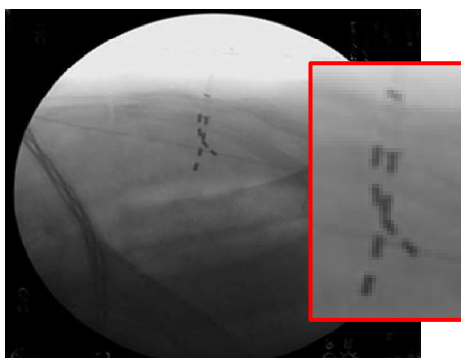
toyobjects_RGB_5objs.bmp



After manipulating label images, verify results.

More Cases??

View Gray-Scale Images as RGB, More Case??



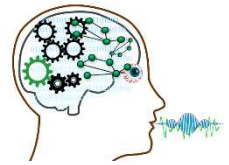


Image Resizing Options

- Anti-aliasing → less jagged edges but more **color artifacts**
 - By weighted average of nearby “*N*” pixels (bilinear 4, **bicubic** 4x4, usual default)
- Another option: **Nearest** → more jagged edges but **NO** color artifacts
 - Faster / simpler = pixel values in the **resized** (output) image are set from an **existing** value of the nearest pixel in the **original** (input) image

`imresize(img, [180 300], 'nearest')`

`cv2.resize(img, dsize=(180, 300), interpolation = cv2.INTER_NEAREST)`

NO “nearest” option (i.e. with anti-aliasing)



With “nearest” option (next slide)

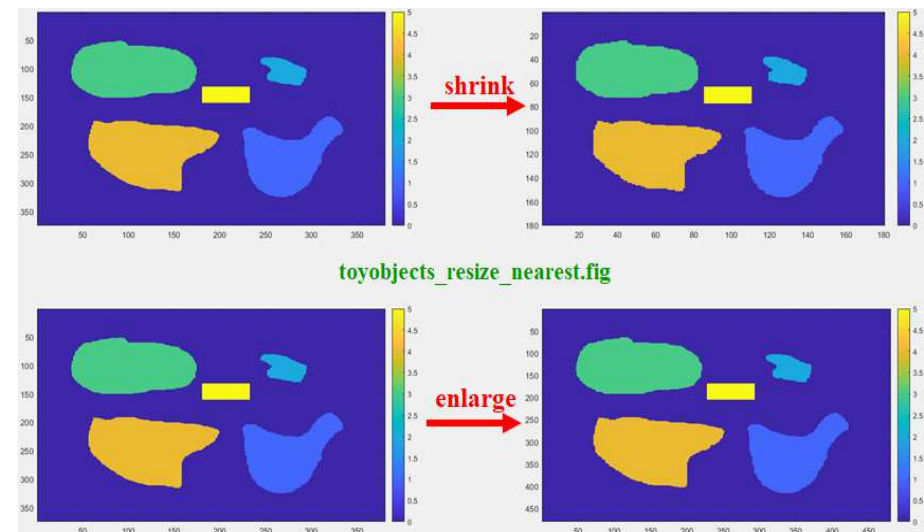
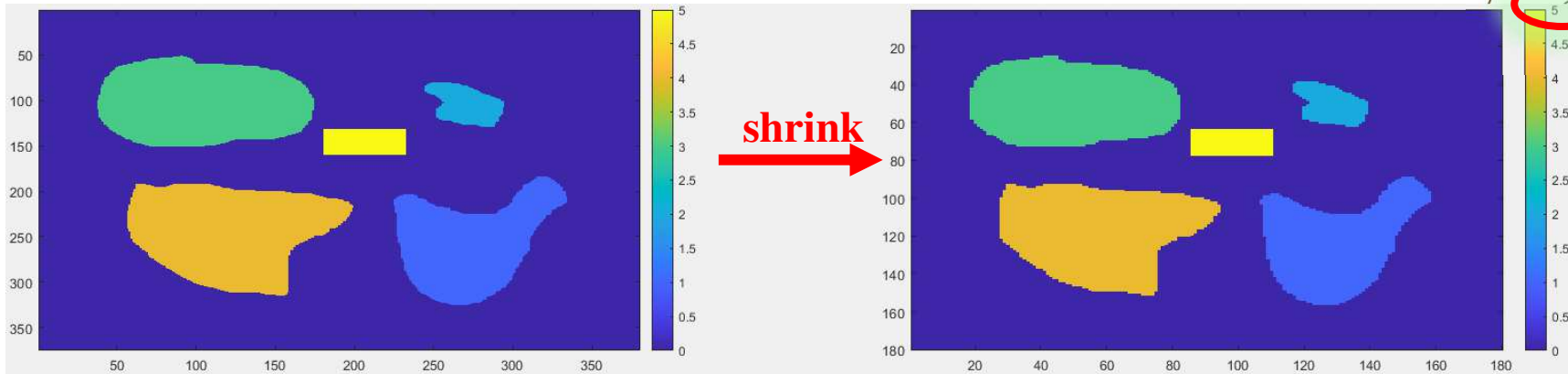
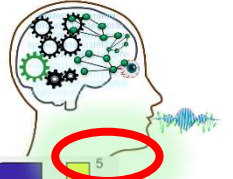
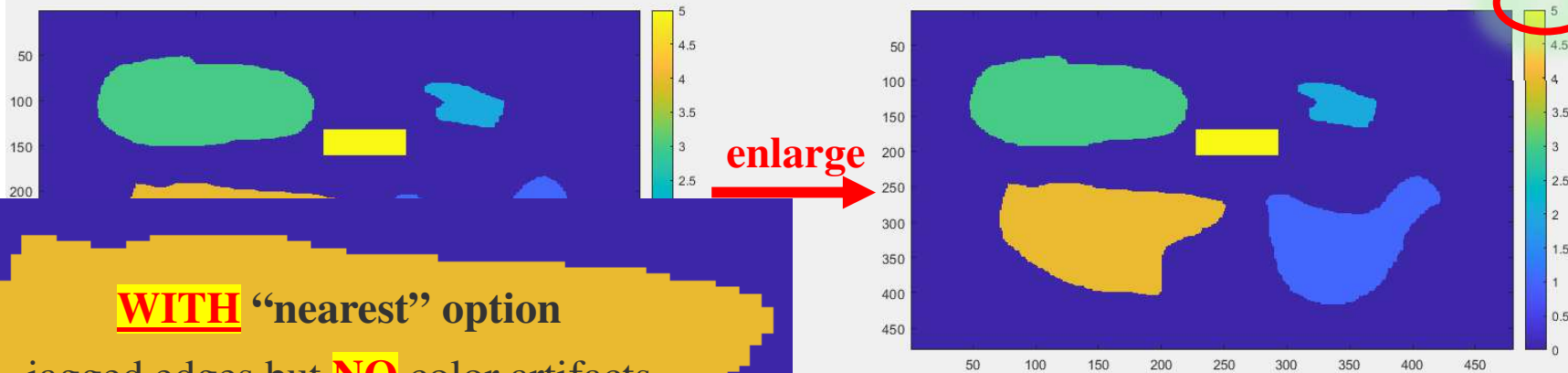


Image Resizing Options Using “*nearest*”



toyobjects_resize_nearest.fig



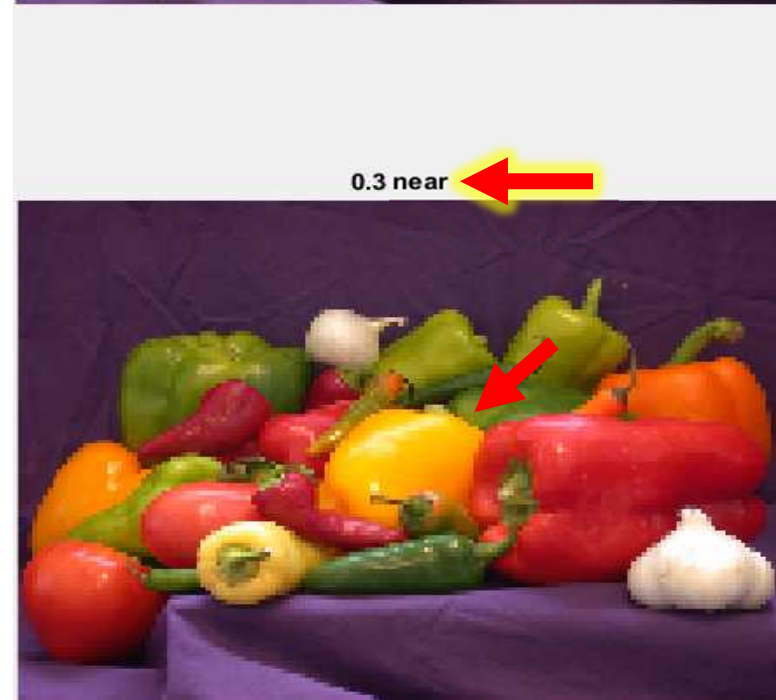
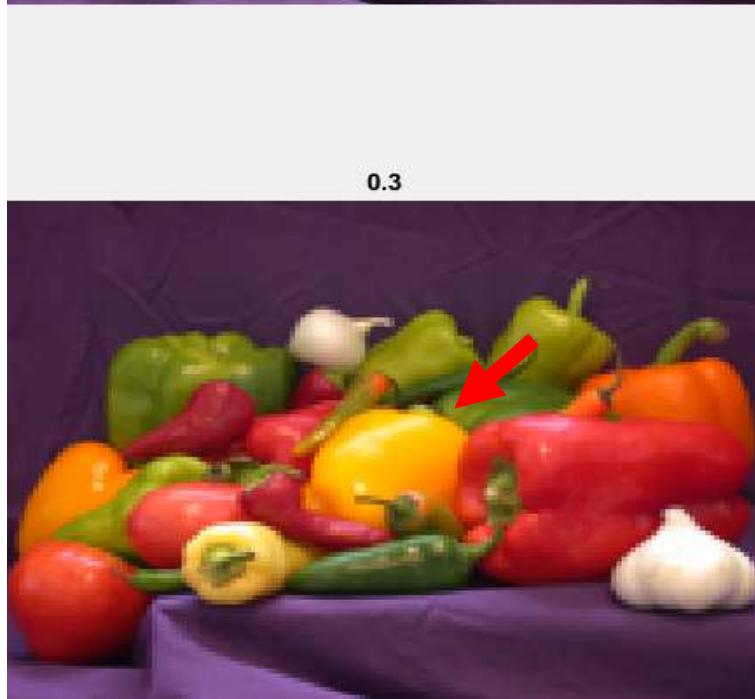
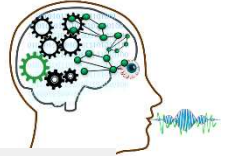
WITH “nearest” option
jagged edges but **NO** color artifacts

NO
nearest
option

`unique(Img(:) )`

`np.unique(Img)`

Image Resizing



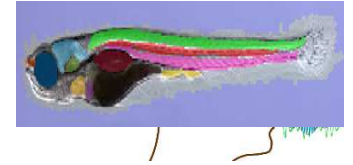
read from BMP & display

Labels / Indexed

Page: 15

Prepare_HW_Data.ipynb

WHY Differences???

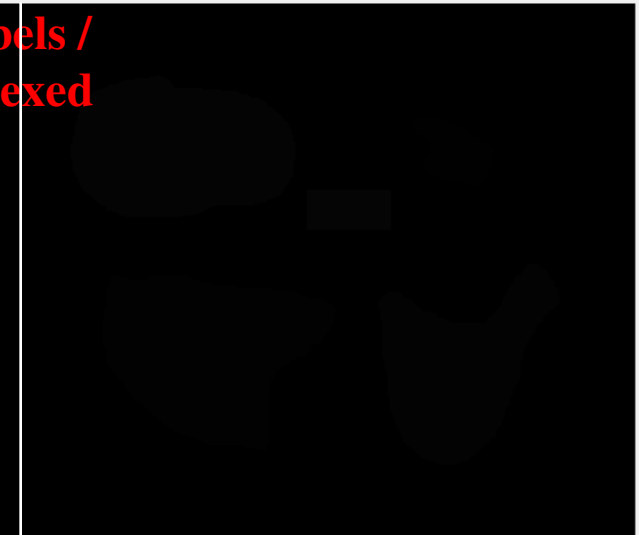


1. add an rectangle, 2. convert to indexed
labeled colors & 3. write to **JPG** & **BMP**

read from JPEG & display

read from BMP & display

Raw, JPEG, BMP

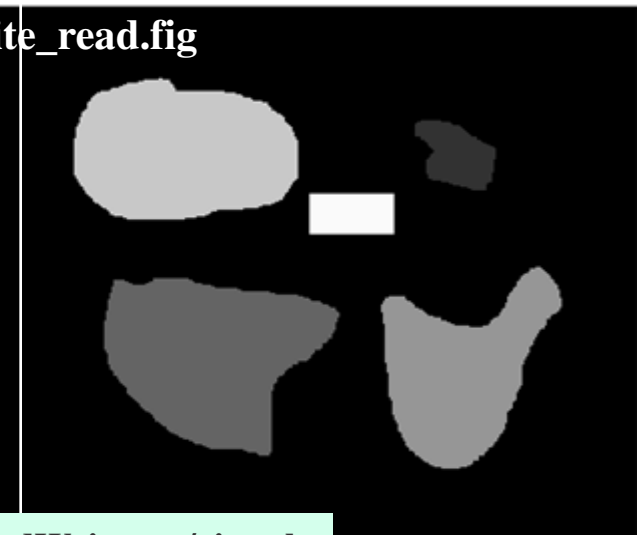
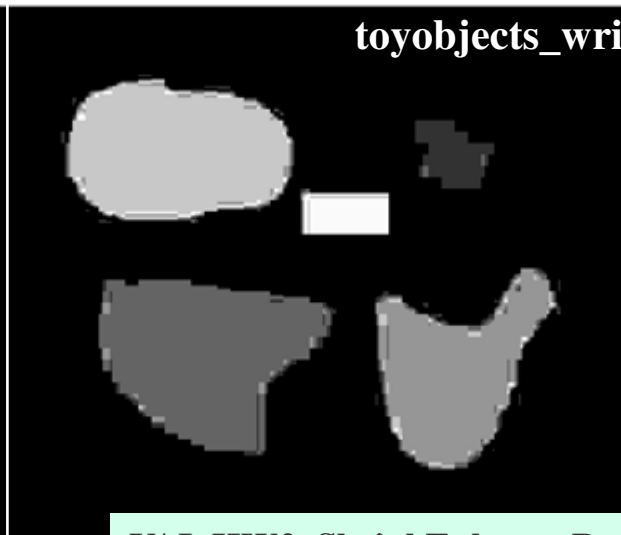


Labels /
Indexed

Folder: \Images, **HW_toyobjects_Gray.bmp**

read from JPEG * 50

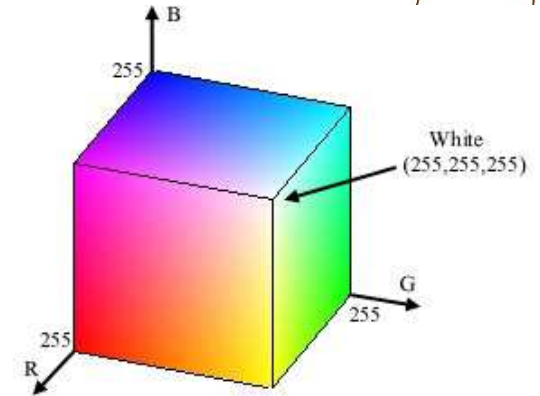
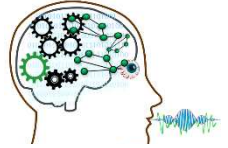
read from BMP * 50



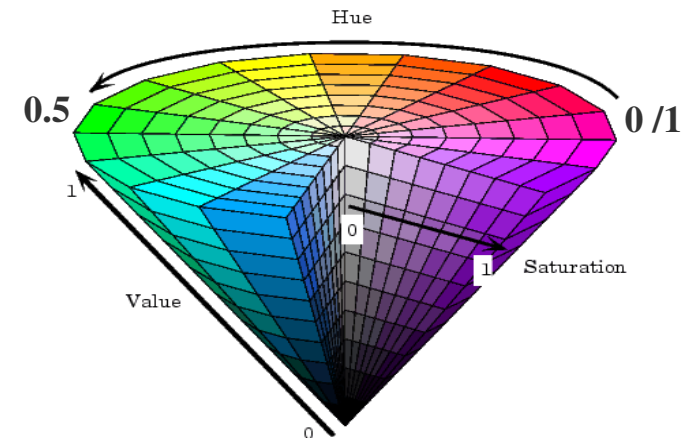
toyobjects_write_read.fig

VAI_HW2_ShrinkEnlarge_ReadWrite.m / .ipynb

Outline

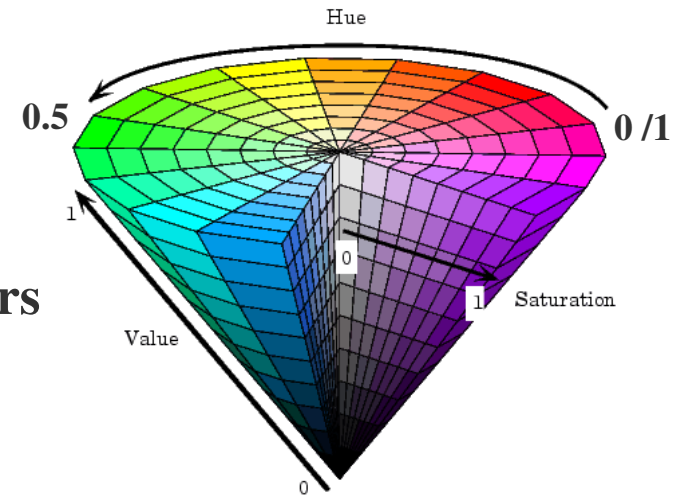


- Color and Color Space
- **RGB** Color Space, Gray-Scale, Black/White
 - Impact of image resizing to colors
- Indexed Colors and Usage
 - Color Motion Map
- Problems / Pitfalls in Dealing with Colors
 - **Dithering / Image Resizing / Saving Images** **Augmentation**
- **HSV** Color Space and Usage
 - Adjust Light Condition, Histogram Equalization

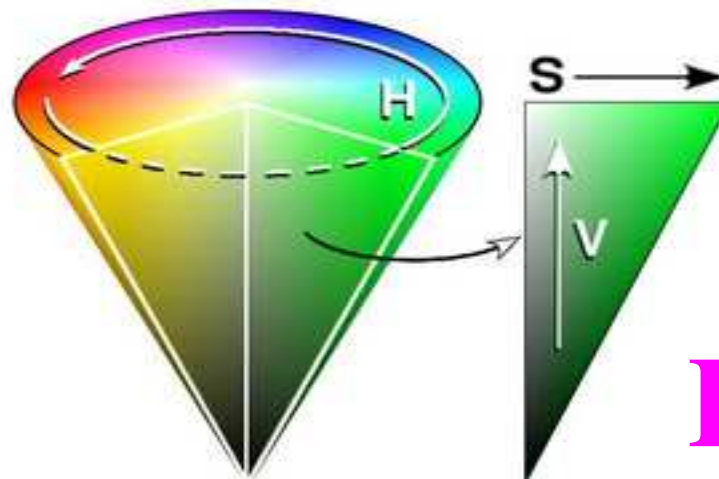
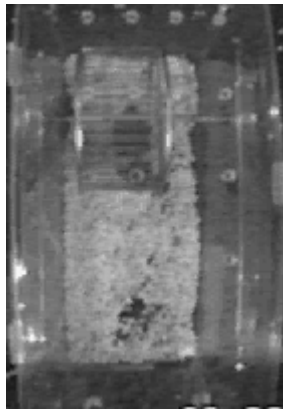


Characteristics of HSV (or HSI)

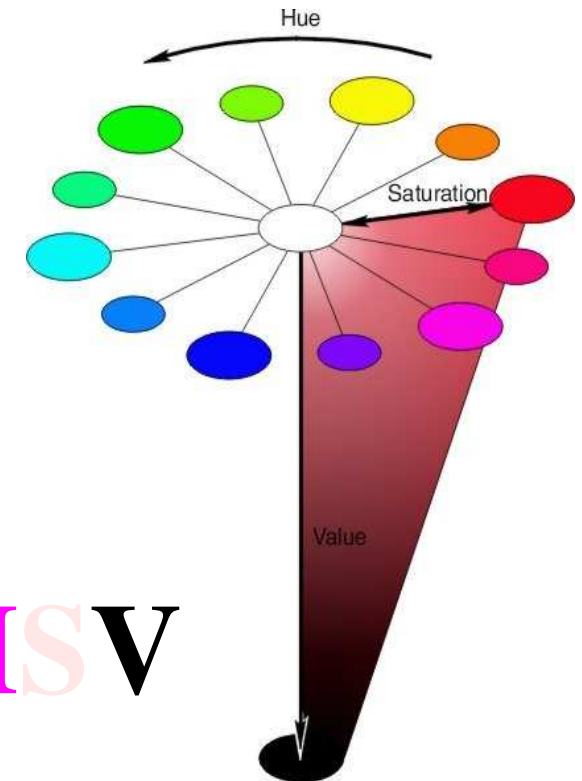
- **Hue**– “color” of the color
- **Saturation**– amount of white diluted with hue / colors
- **Brightness**– volume / intensity of light (luminance)
- HSV / HSI is useful for image processing / **augmentation**
 - **Histogram equalization** is applied to **Intensity**
 - Infrared video has **NO** meaningful color info. **ONLY** **light**



Infrared Video

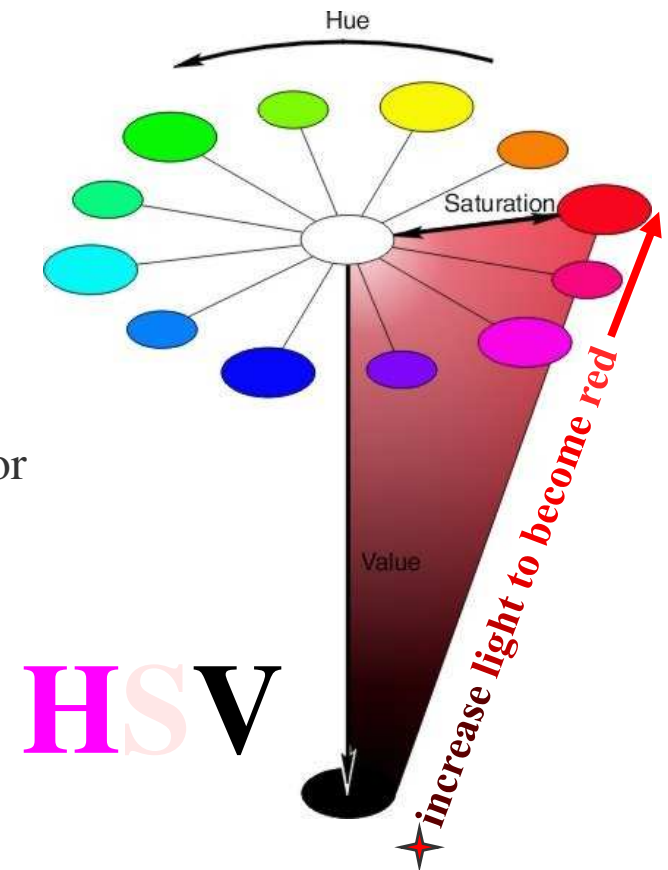
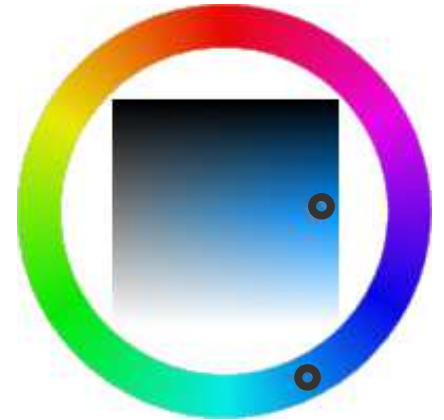


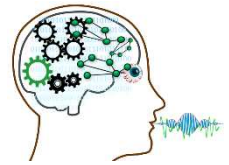
HSV



Properties of HSV Colors

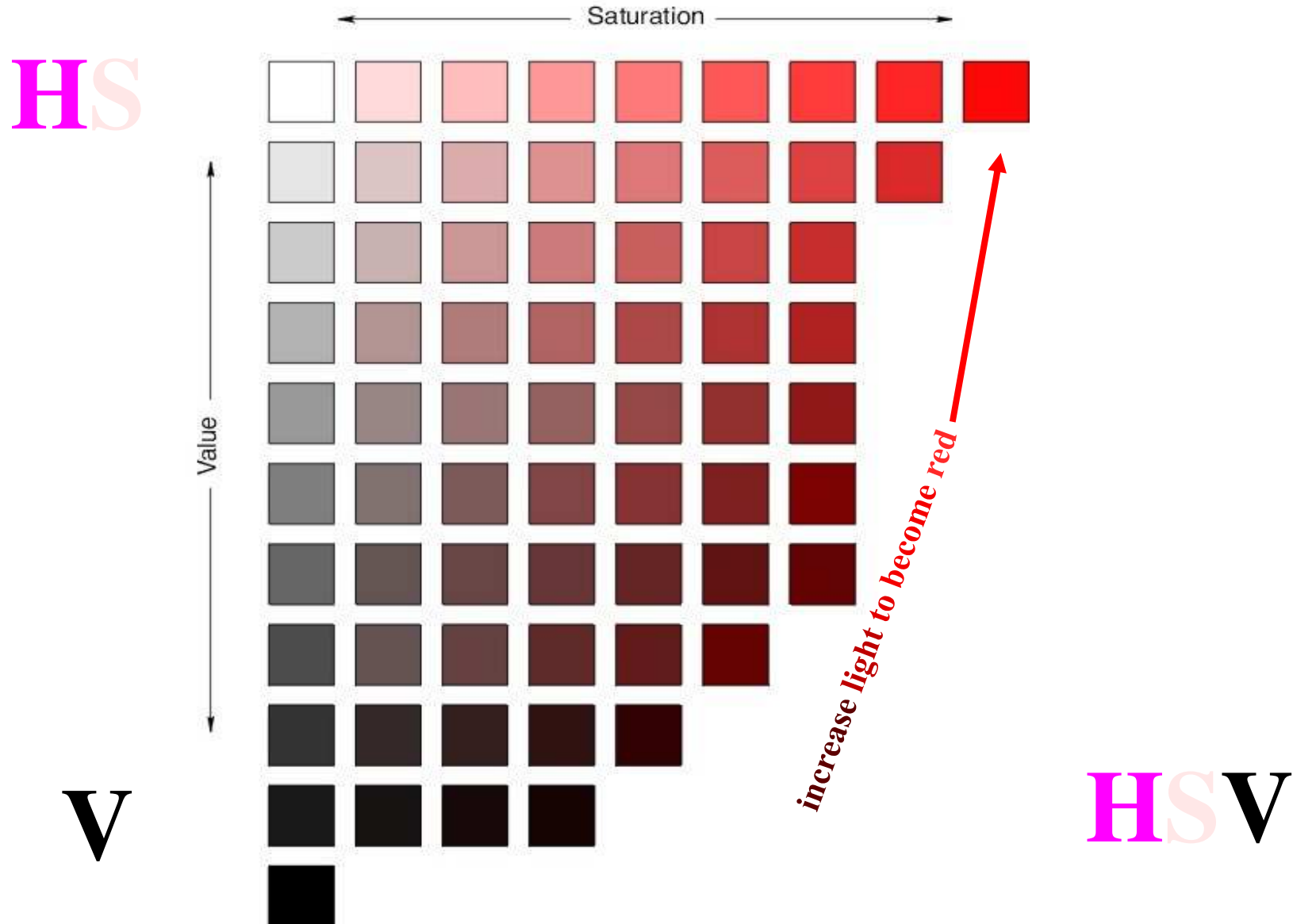
- Hue (or color):
 - Visual sensation of perceived color
- Saturation (or purity of real color):
 - Colorfulness judged in proportion to white color
 - A pure color has a saturation of 100%
 - The saturation of white or gray light is zero
 - $\text{Saturation} = (\text{brightness} - \min(R, G, B)) / \text{brightness}$
- Luminance (or light or brightness):
 - Sensation of which area emits more or less light
 - Our eyes do **NOT** detect brightness linearly with color
 - we see Green as brighter than Blue
 - Luminance value =
 $0.3 \cdot R + 0.59 \cdot G + 0.11 \cdot B = \max(R, G, B)$



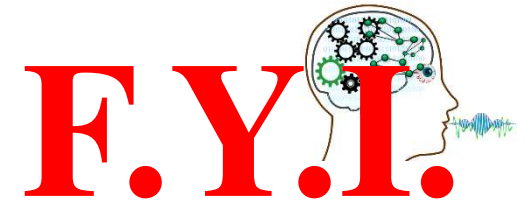


Saturation and Brightness (S & V) Variations

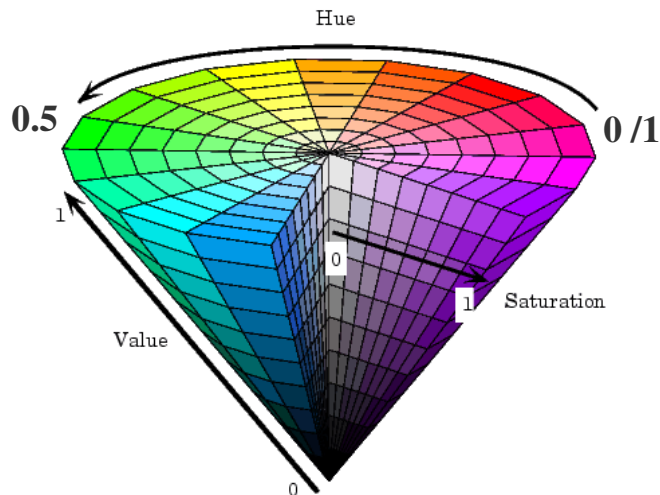
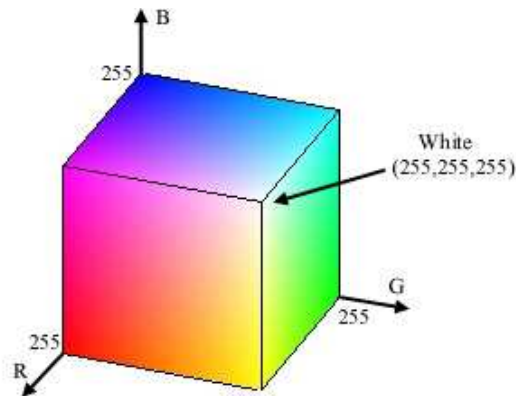
Example saturation and brightness variations on a single red hue



RGB $\Leftrightarrow$ HSV Formula, FYI Only



- Programming (single function call) in next slides...



$$C_{max} = \max(R, G, B), \quad C_{min} = \min(R, G, B)$$

$$\Delta = C_{max} - C_{min}$$

$$H = \begin{cases} 0 & \Delta = 0 \\ 60^\circ \times \left(\frac{G-B}{\Delta} \bmod 6 \right) & C_{max} = R \\ 60^\circ \times \left(\frac{B-R}{\Delta} + 2 \right) & C_{max} = G \\ 60^\circ \times \left(\frac{R-G}{\Delta} + 4 \right) & C_{max} = B \end{cases}$$

$$S = \begin{cases} 0 & C_{max} = 0 \\ \frac{\Delta}{C_{max}} & \text{otherwise} \end{cases}$$

$$V = C_{max}$$

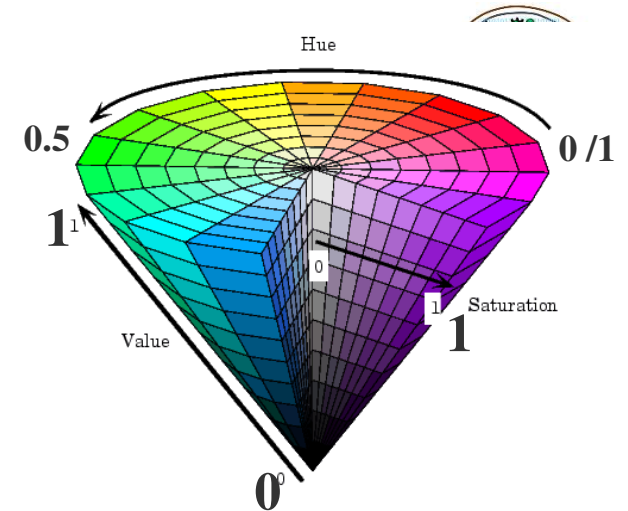
h = rgb2hsv(img);

opencv

cv2.cvtColor(RGB_image, cv2.COLOR_BGR2HSV)

cv2.cvtColor(HSV_img, cv2.COLOR_HSV2BGR)

RGB $\Leftrightarrow$ HSV



- **RGB** : [int, int, int] **or** [double, double, double]
 - 0 .. 255 **OR** 0 .. 1
- **HSV**: [double, double, double]
 - ALWAYS in 0 .. 1

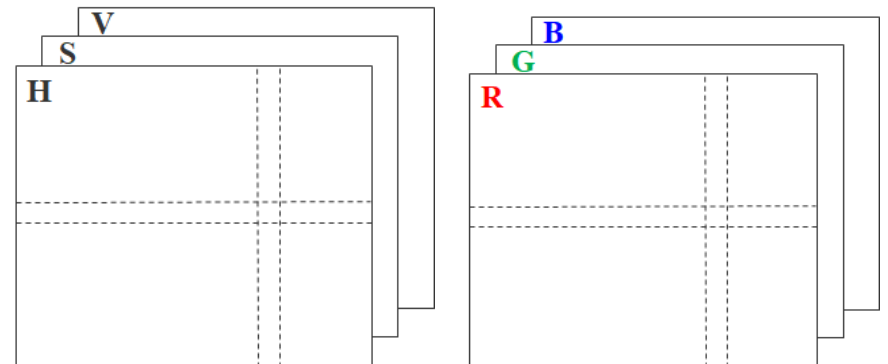
- **RGB $\Leftrightarrow$ HSV**
 - `hsv_image = rgb2hsv(rgb_image);`
 - `rgv_image = hsv2rgb(hsv_image);`

opencv

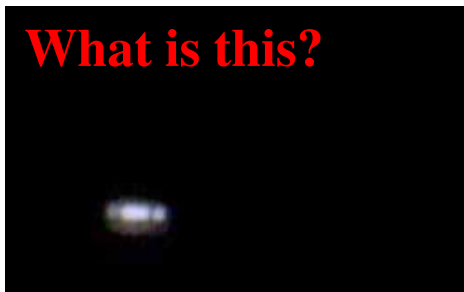
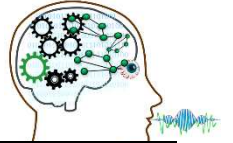
```
cv2.cvtColor(RGB_image, cv2.COLOR_BGR2HSV)
```

```
cv2.cvtColor(HSV_Img, cv2.COLOR_HSV2BGR)
```

```
img = imread('parrots.jpg');  
h = rgb2hsv(img);  
imshow(h)  
figure, imshow(hsv2rgb(h))
```



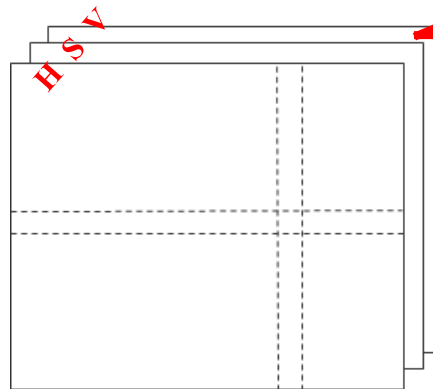
HSV App 1, Using Intensity to Identify Lights



What is this?

```
H = rgb2hsv(i);  
imshow(H(:,:,3))
```

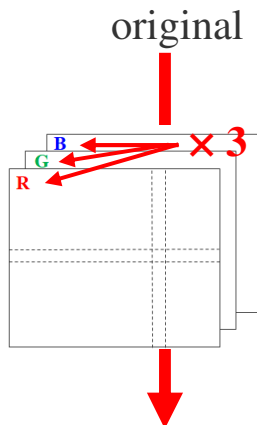
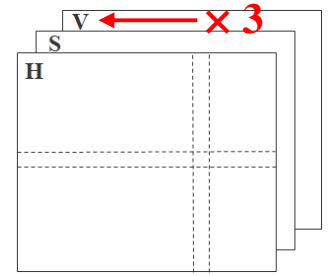
```
H = cv2.cvtColor(RGB_image, cv2.COLOR_BGR2HSV)  
cv2.imshow("V Channel", H[:, :, 2])
```



HSV App 2, Manipulation of Brightness in Color Images

```
i = imread('darkbuilding.jpg');  
figure, imshow(imadjust(i, [.2 .3 0; .6 .7 1], []))
```

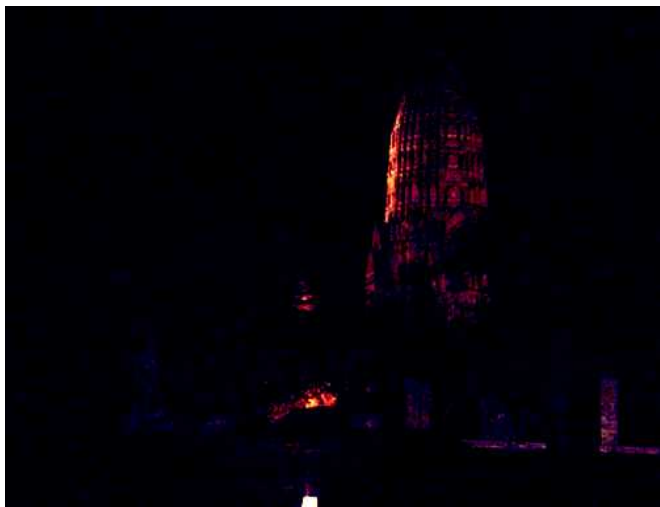
```
h = rgb2hsv(i);  
hb = h;  
hb(:, :, 3) = hb(:, :, 3) * 1.5;  
figure, imshow(hsv2rgb(hb))
```



RGB * 3
= adjust
based on
R, G, B
colors
together



brightness
× 1.5
(:, :, 3) * 1.5
h[..., 2] *= 1.5



brightness
× 3
(:, :, 3) * 3
h[..., 2] *= 3

HSV App 2, Adjust **Intensity** (HSV) for Dark Images, Check Details...

- Manipulate (**multiplying**) intensity of HSV

adjust **RGB** colors



(a) (b)
(c) (d)



(a) Original RGB
(c) brightness $\times 2$



(b) adjust RGB $\times 3$
(d) brightness $\times 5$



(a) (b)
(c) (d)



(a) Original RGB
(c) brightness $\times 2$



(b) adjust RGB $\times 3$
(d) brightness $\times 5$

straightforward brightness adjustment

BUT, may not be the best way

HSV App 3, Better Solution: Histogram Equalization on HSV



(a) (b)

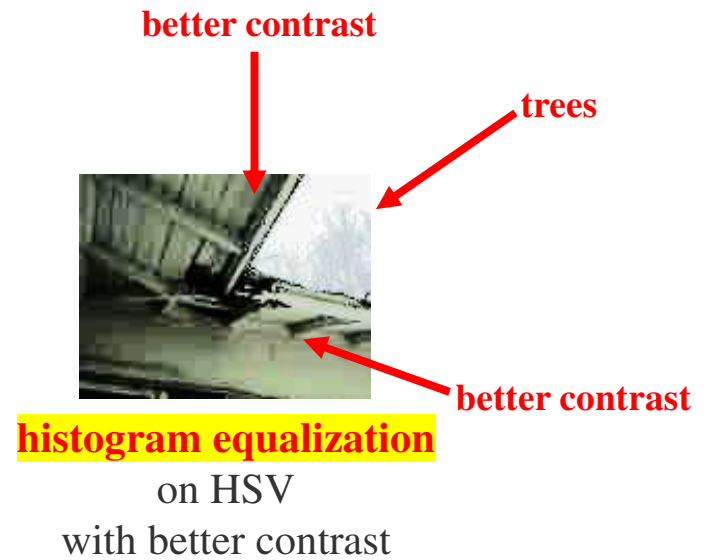
(c) (d)

(a) Original RGB

(b) adjust RGB

(c) brightness $\times 2$

(d) brightness $\times 5$

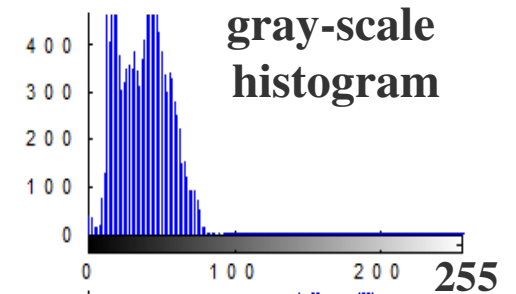
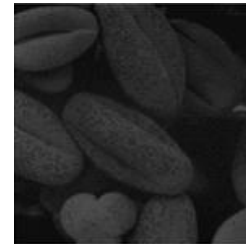


COLOR Histogram and Equalization

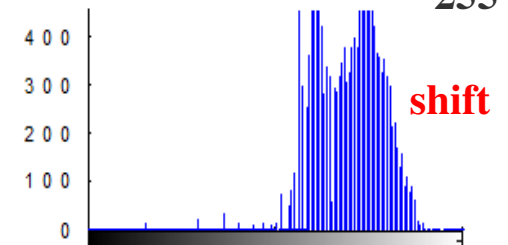
`histeq(image)`

but have to convert to gray first
`cv2.equalizeHist(image)`

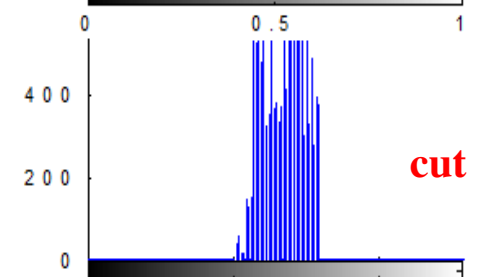
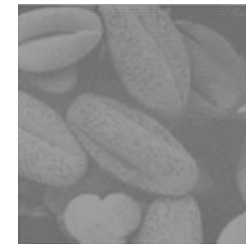
too
dark



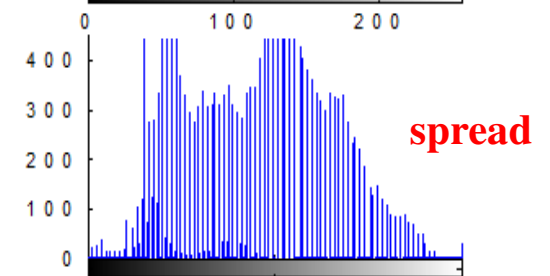
brightness
+ 0.6
too
bright



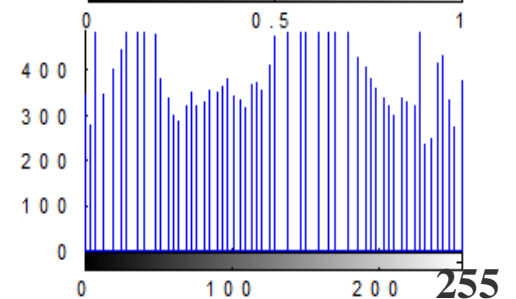
low
contrast



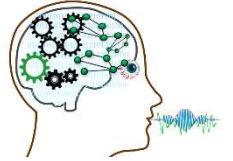
brightness
× 3



histogram
equalization



Histogram Equalization on Medical Image



Input



Another Example

very dark



histeq



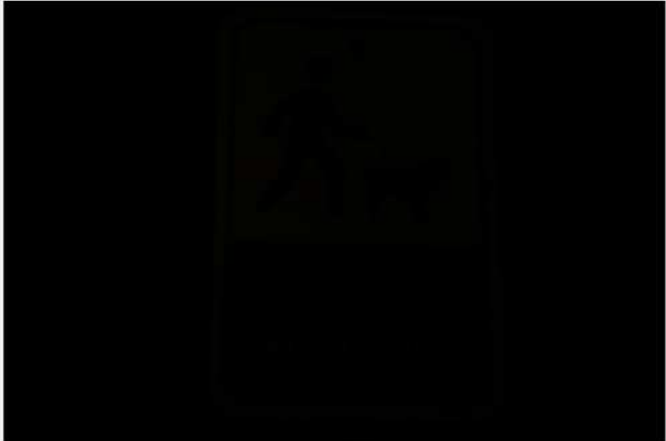
Lung_histeq.fig

Δ of Input - histeq

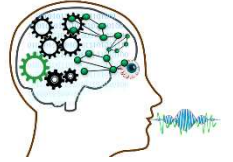


Recover Low-Light Images Using Deep Learning



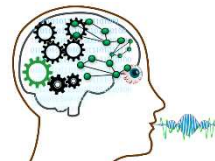
Short Exposure Test Image s Exposure Time = 0.04 s 	Input	Ground Truth	Long Exposure Target Image s Exposure Time = 10 s 
Prediction Time Good CPU = 150 seconds GPU = 1.9 seconds			
Low-Light Recovery Network Prediction 	Recover deep learn	Recover histeq	Histogram Equalization 

<https://www.mathworks.com/help/images/learning-to-see-in-the-dark.html>



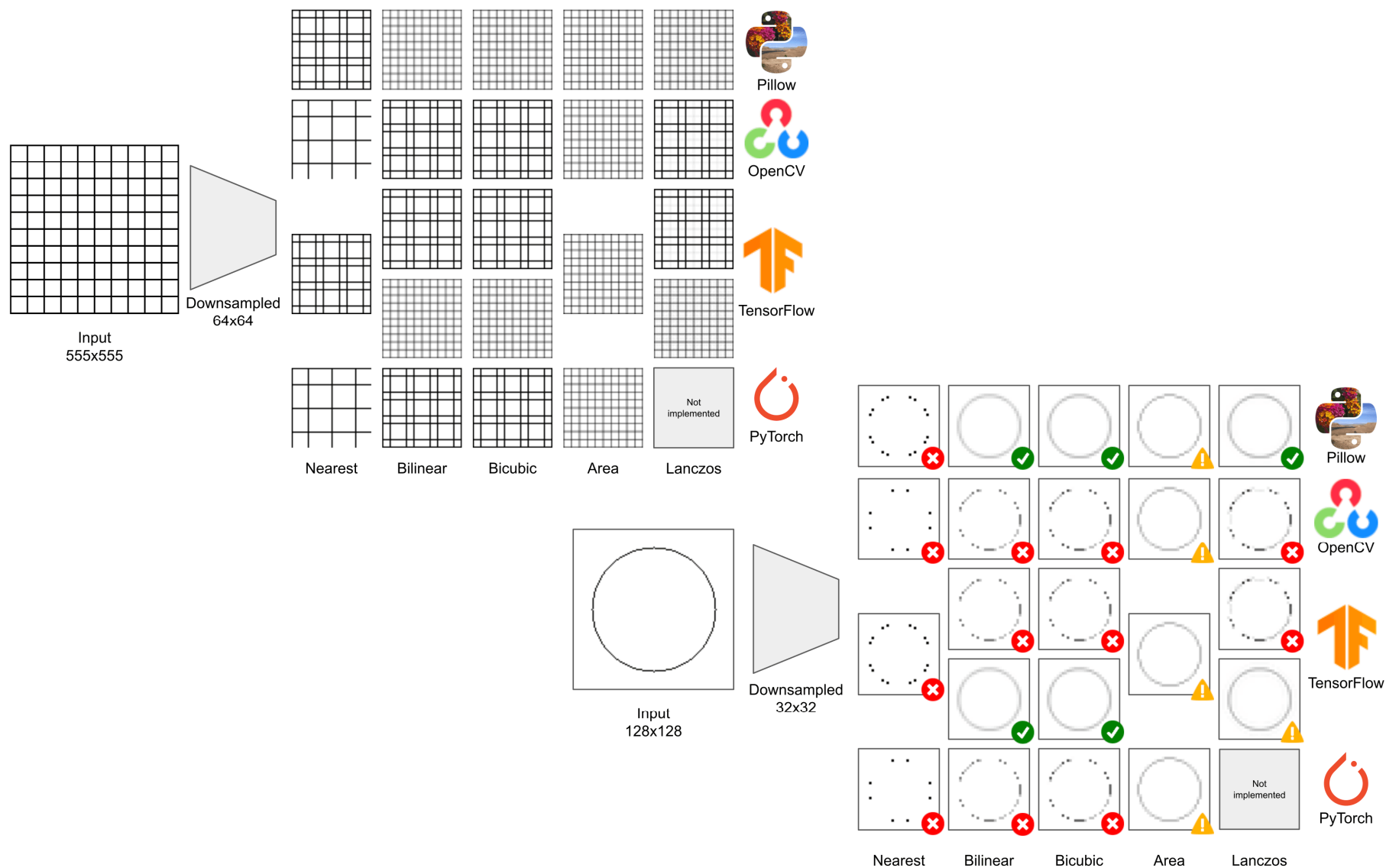
Appendix

- **Adjust color / grayscale images**
- **Histogram equalization details**



The Dangers Behind Image Resizing

- <https://zuru.tech/blog/the-dangers-behind-image-resizing>



Adjust Brightness in Gray Images

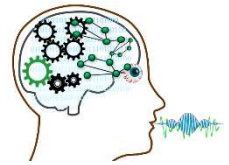


```
>> i= imread('hmatface.jpg');  
>> imshow(i);  
>> figure,imshow(imadjust(i, [0.2 0.8], []))  
>> figure,imshow(imadjust(i, [0.2 0.8], [0.1 0.7]))
```

%% dark → darker, bright → brighter

%% drak → darker, bright → darker





Histogram Equalization– Transforming Images

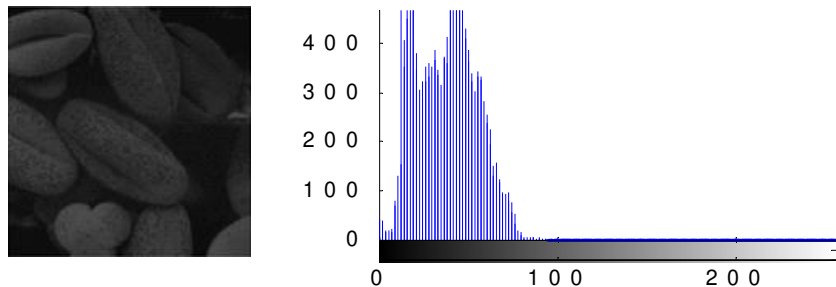
■ Objective➔

- Produce an image with *approximately* equal # of pixels in all grey levels
- Remaps pixel values to approximately evenly distribute values
- Try to use **ALL** available gray levels in an output image

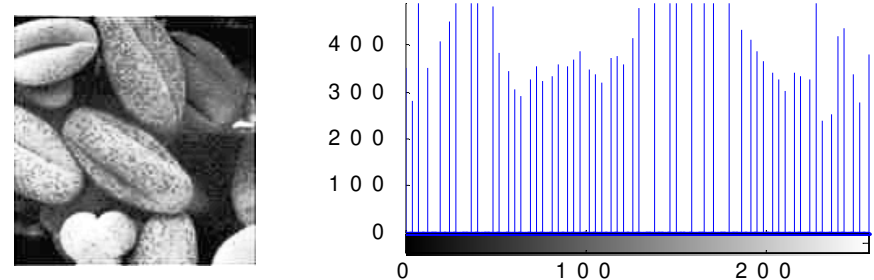
■ Applications ➔

- Make sure a set of related images have *close (similar) histograms*
- A simple but effective way of **bringing out image detail** for naked eyes

dark

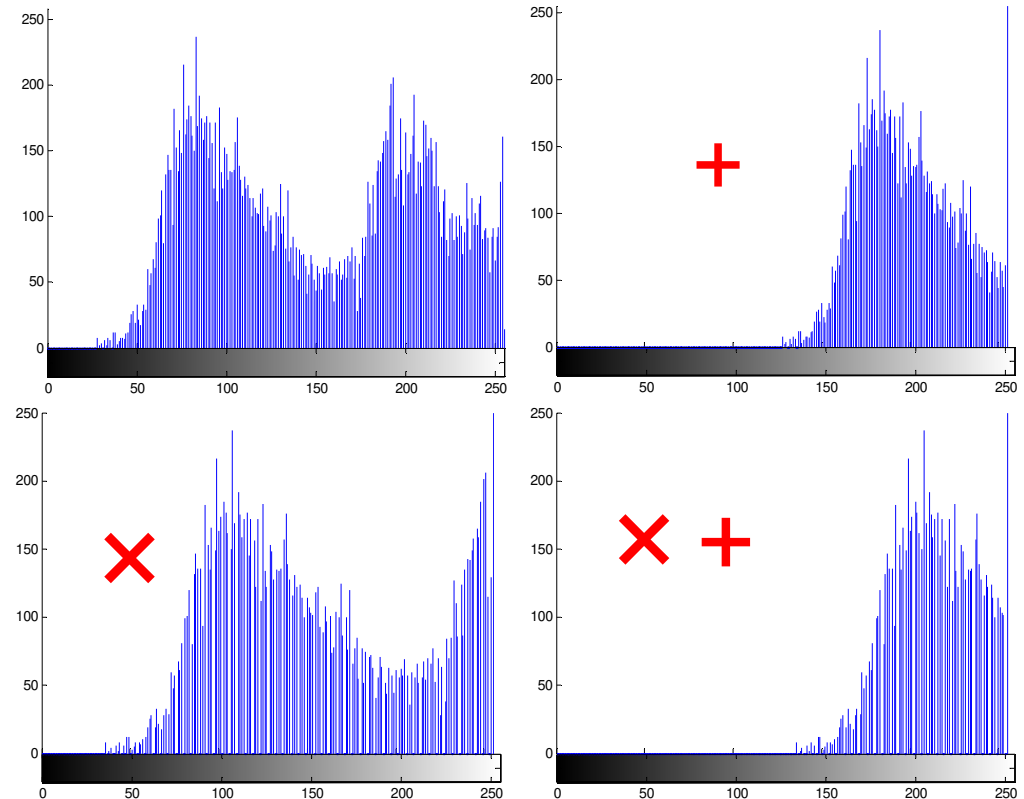
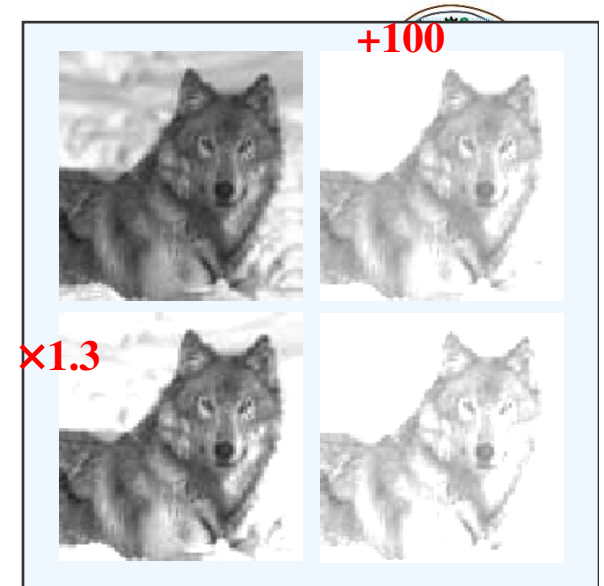


histogram equalization



Histogram and Linear Mapping

- Adjusting brightness / contrast of gray images
 - Adjusting brightness by adding a constant
 - Adjusting contrast by multiplying a constant
- Adding a constant shift a histogram
 - Without changing the remaining histogram shape
- Multiplying a constant spread/compress histogram



$$Output(x, y) = a \times Input(x, y) + b$$

Histogram Examples

```
>> i=imread('coffeebins.jpg');
```

```
>> g=rgb2gray(i);
```

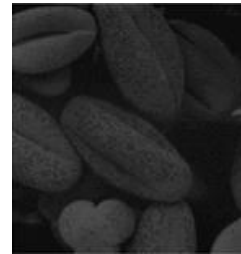
```
>> h=rgb2hsv(i);
```

```
>> bh = h;
```

```
>> bh(:, :, 3) = bh(:, :, 3) + 0.6;
```

```
>> b = rgb2gray(hsv2rgb(bh));
```

too dark



**brightness + 0.6
too bright**



```
>> [counts, x] = imhist(rgb2gray(i));
```

```
>> max(find(counts))
```

```
ans = 98
```

```
>> tmp = counts(1:50);
```

```
>> counts = 0;
```

```
>> counts(101:150) = tmp;
```

```
>> counts(151:256)=0;
```

```
>> l=histeq(g, counts);
```

```
>> bbh = h;
```

```
>> bbh(:, :, 3) = bbh(:, :, 3) * 3;
```

```
>> bb = rgb2gray(hsv2rgb(bbh));
```

```
>> imshow(histeq(g))
```

```
>> imhist(histeq(g))
```

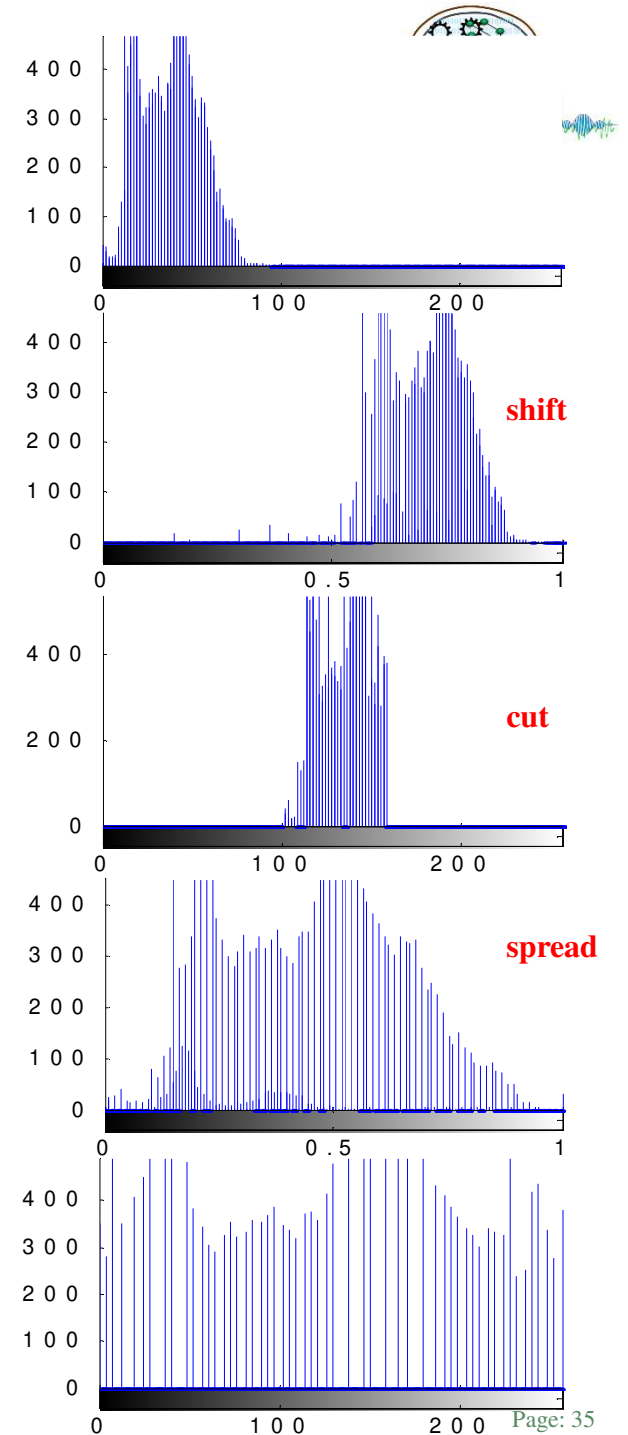
low contrast



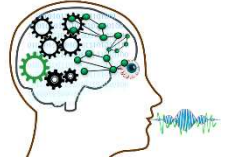
brightness × 3



histogram equalization



Histogram Equalization– Non-linear Mapping



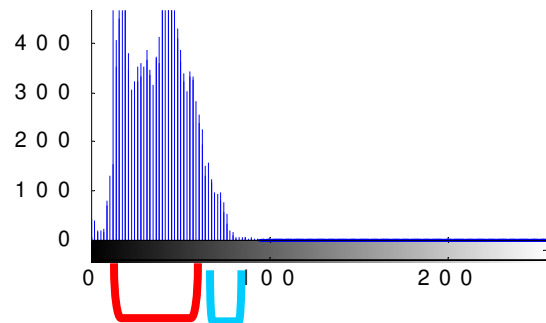
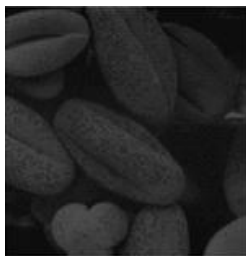
- Produce an image with
 - Approximately equal # of pixels in all grey levels while preserve histogram shape
 - Try to use **ALL** available gray levels in an output image

$$\text{Output}(x, y) = \textcolor{red}{a} \times \text{Input}(x, y) + b$$

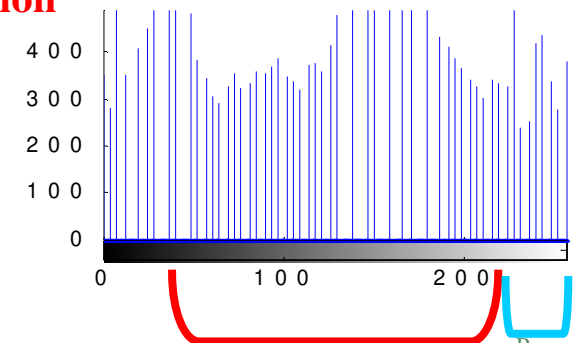
- Histogram equalization is a non-linear mapping
 - More grey levels (contrast, $a > 1$) set to where there are more pixels
 - Fewer grey levels (contrast, $a < 1$) set to where there are fewer pixels
- How???

- *Cumulative sum* of histogram has these features!!!

too dark



histogram equalization



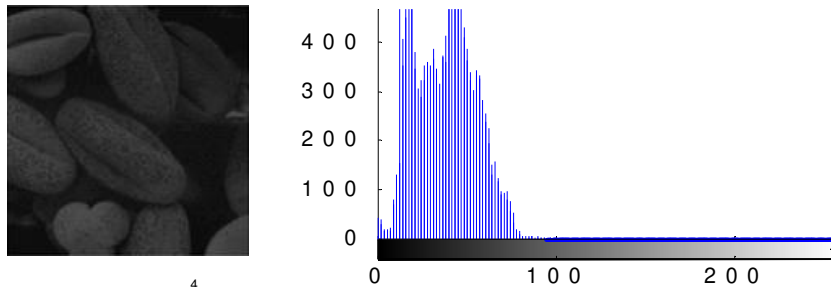
check the histogram shapes

Cumulative Sum & Histogram Equalization

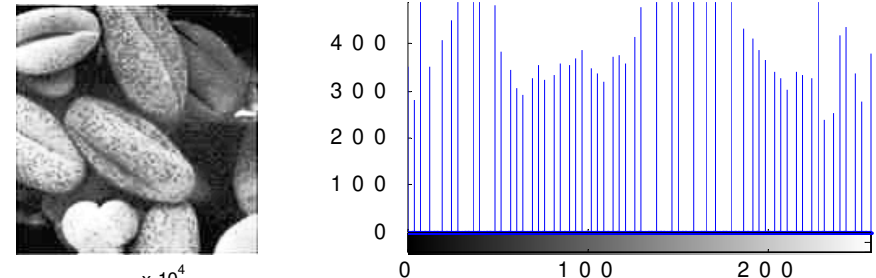
$$\text{Output}(x, y) = a \times \text{Input}(x, y) + b$$

- More grey levels (contrast, $a > 1$) set to where there are more pixels
- Fewer grey levels (contrast, $a < 1$) set to where there are fewer pixels

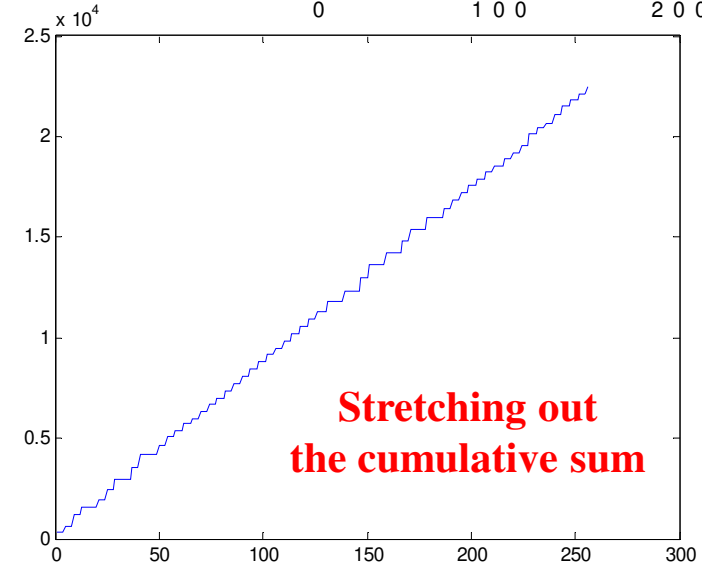
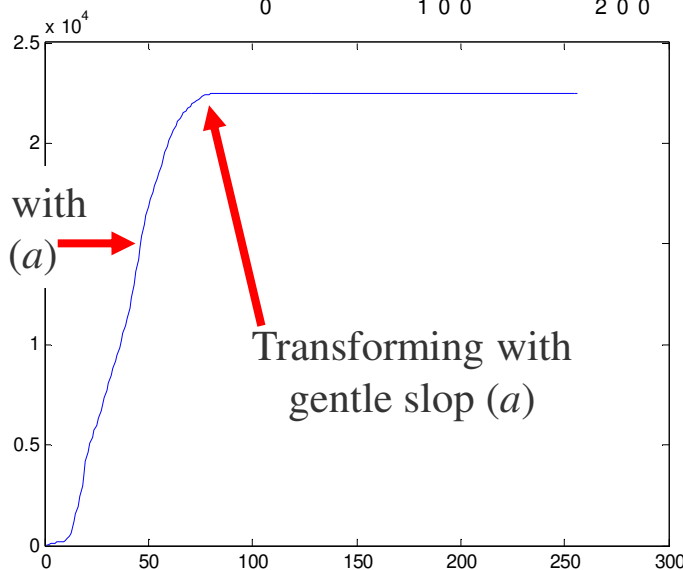
dark



histogram equalization



Transforming with
greater slop (a)

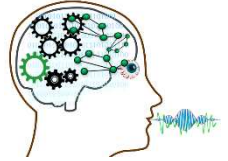


■ Resulting in a stretched out cumulative sum

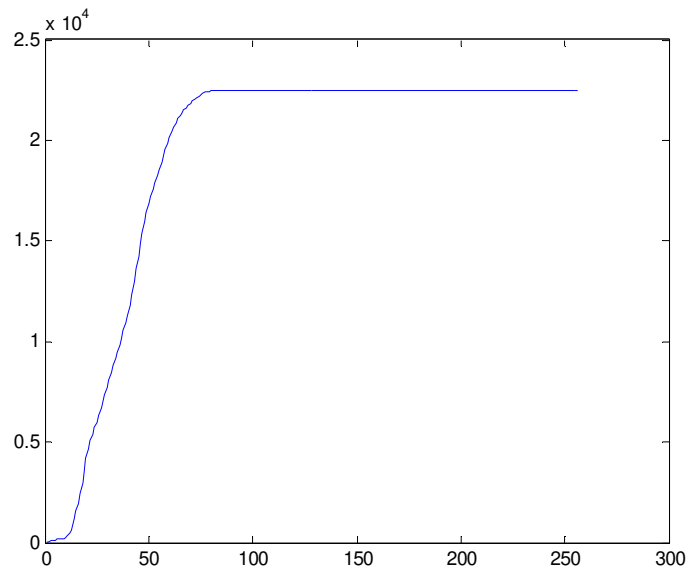
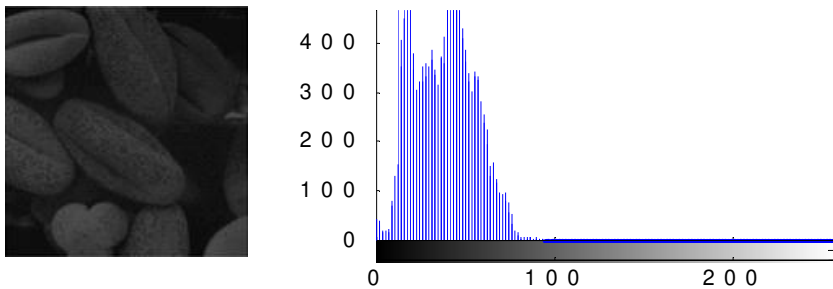
- This is histogram equalization

Computing Cumulative Sum in MatLab

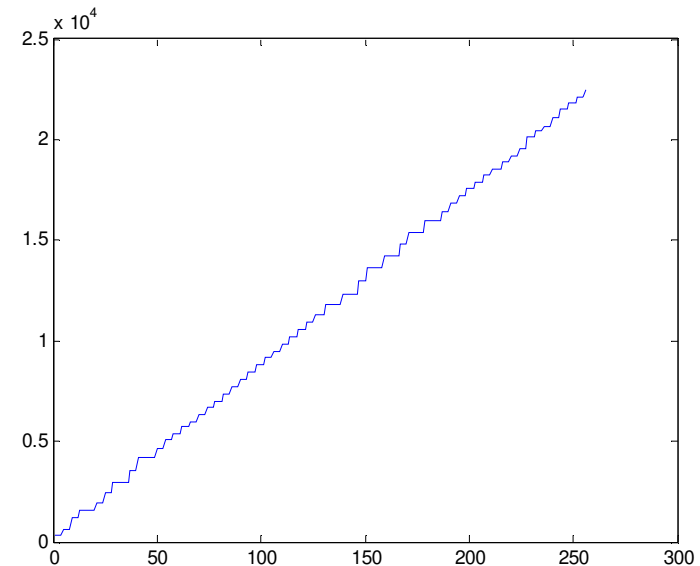
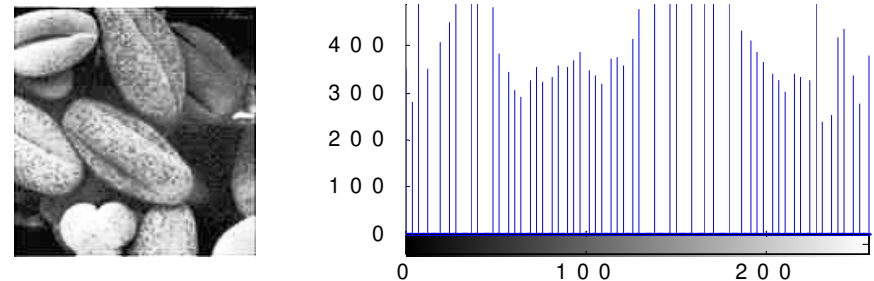
```
>> i = imread('coffeebins.jpg');
>> g = rgb2gray(i);
>> [counts, x] = imhist(g);
>> c = cumsum(counts);
>> plot(c);
%% normalized = c / sum(counts)
```



dark

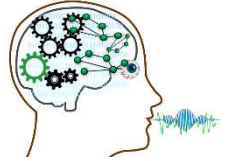


histogram equalization



```
>> f=255/numel(g), ff=c*f; %% normalization
>> newi=zeros(size(g));for idx=0:255, newi(find(g==idx))=ff(idx+1);end % mapping
>> imshow(uint8(newi)), [ccc xxx]=imhist(uint8(newi)); plot(cumsum(ccc))
```

Histogram Equalization Steps



- Remaps pixel values to approximately evenly distribute values
 - 1) Calculate cumulative histogram
 - 2) Normalize cumulative histogram by dividing the total # of pixels (= % of intensity values)
 - 3) Multiply normalized values by the maximum gray-level values, → decide gray value
 - 4) Map the original values to the results obtained in step 3

Pseudo algorithm

Get histogram H

Calculate the cumulative histogram Hc

$$Hc[0] = H[0]$$

$$Hc[i] = Hc[i-1] + H[i]$$

Define a remapping (and normalize)

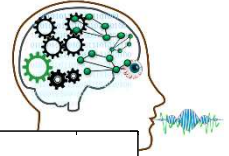
$$T(i) = \text{round}((\text{Max_GrayLevel} / NM) * Hc[i]) \quad \text{or} \quad \text{round}((Hc[i] / NM) * \text{Max_GrayLevel})$$

Where N is the width of the image and M is the height

Remap the pixel values

$$\text{newval} = T[\text{oldval}]$$

Histogram Equalization Example



Get histogram H

Calculate the cumulative histogram H_c

$$H_c[0] = H[0] \text{ \& } H_c[i] = H_c[i-1] + H[i]$$

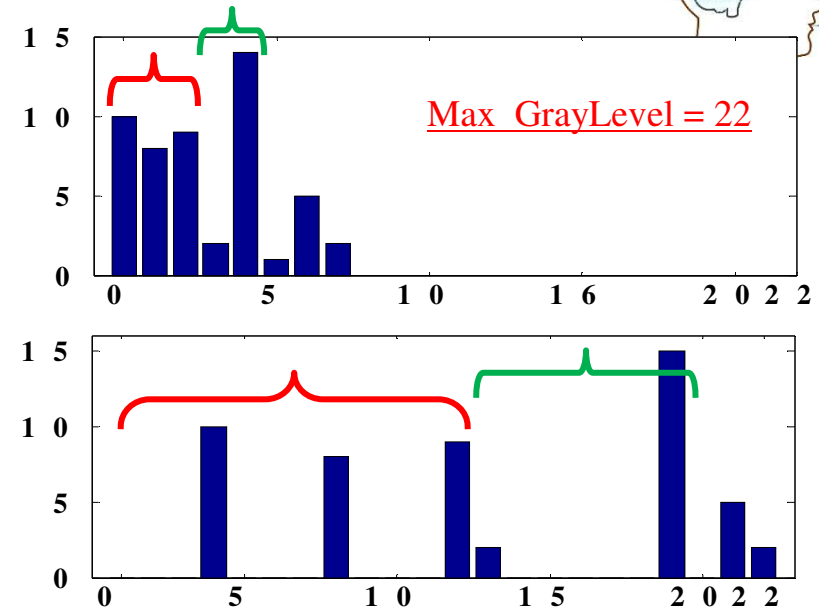
Define a remapping (and normalize)

$$T(i) = \text{round}((\text{Max_GrayLevel} / NM) * H_c[i]) \text{ or } \text{round}((H_c[i] / NM) * \text{Max_GrayLevel})$$

N is the width of the image and M is the height

Remap the pixel values

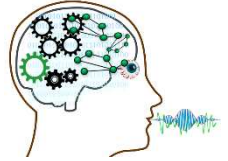
$$\text{newval} = T[\text{oldval}]$$



More grey levels (contrast, $a > 1$) are assigned to where there are more pixels
Fewer grey levels (contrast, $a < 1$) are assigned to where there are fewer pixels

Original Gray-Level	Number of pixels	Step 1 cumulative histogram	Step 2 & 3 normalized cumulative distribution	Mapped Gray-Level	Number of pixels
0	10	10	$(10 / 51) * 22 = 0.20 * 22 \approx 4$	4	10
1	8	18	$(18 / 51) * 22 = 0.33 * 22 \approx 8$	8	8
2	9	27	$(27 / 51) * 22 = 0.53 * 22 \approx 12$	12	9
3	2	29	$(29 / 51) * 22 = 0.57 * 22 \approx 13$	13	2
4	14	43	$(43 / 51) * 22 = 0.84 * 22 \approx 19$	19	15
5	1	44	$(44 / 51) * 22 = 0.86 * 22 \approx 19$	19	
6	5	49	$(49 / 51) * 22 = 0.96 * 22 \approx 21$	21	5
7	2	51	$(51 / 51) * 22 = 1 * 22 \approx 22$	22	2

Histogram Equalization Example and Cumulative Sum



(Max GrayLevel = 22)

```
>> a=zeros(1, 22); b=zeros(1,22);
```

```
>> a(1:8)=[10 18 27 29 43 44 49 51];
```

%% cumulative sum

```
>> a(9:end)=51;
```

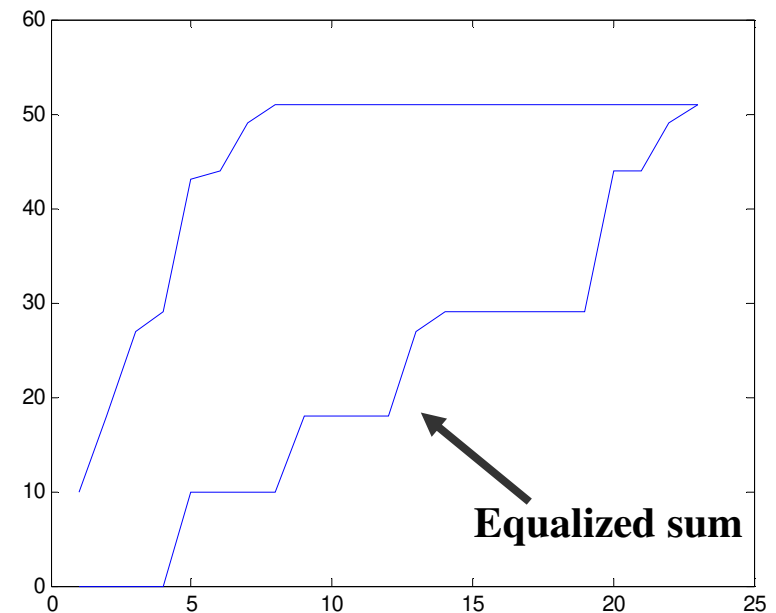
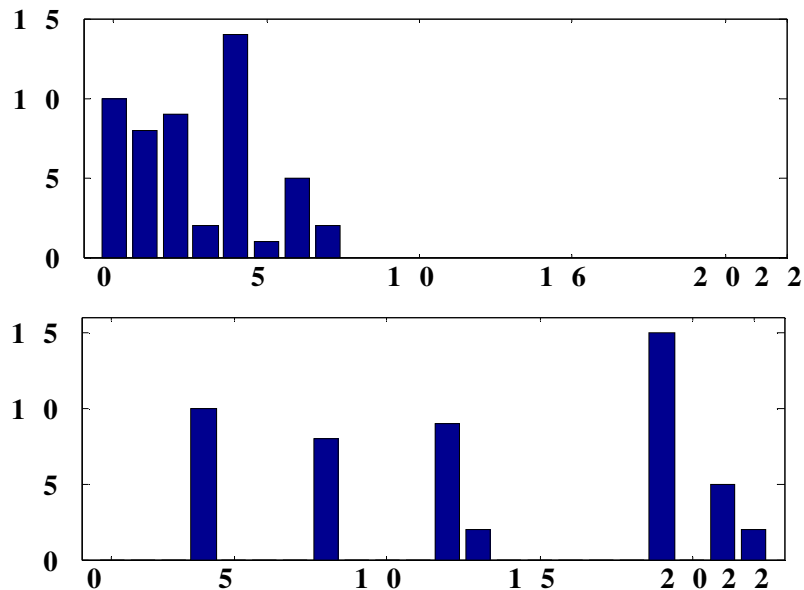
%% cumulative sum

```
>> plot(a)
```

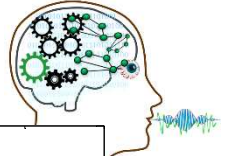
```
>> b(4:7)=10;b(8:11)=18;b(12)=27;b(13:18)=29;b(19:20)=44;b(21)=49;b(22)=51;
```

%% equalized

```
>> plot(a), hold on, plot(b)
```



Histogram Equalization Example (Max_GrayLevel = 7)



Get histogram H

Calculate the cumulative histogram H_c

$$H_c[0] = H[0] \text{ \& } H_c[i] = H_c[i-1] + H[i]$$

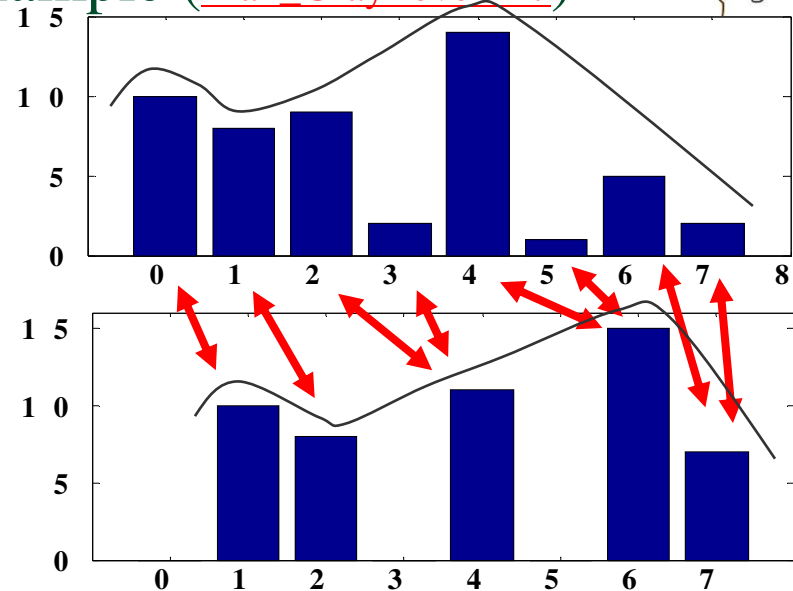
Define a remapping (and normalize)

$$T(i) = \text{round}((\text{Max_GrayLevel} / NM) * H_c[i]) \text{ or } \text{round}((H_c[i] / NM) * \text{Max_GrayLevel})$$

N is the width of the image and M is the height

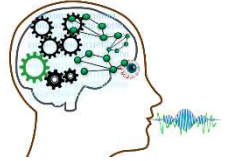
Remap the pixel values

$$\text{newval} = T[\text{oldval}]$$



Original Gray-Level	Number of pixels	<u>Step 1</u> cumulative histogram	<u>Step 2 & 3</u> normalized cumulative distribution	Mapped Gray-Level	Number of pixels
0	10	10	$(10 / 51) * 7 = (7 / 51) * 10 \approx 1$	1	10
1	8	18	$(18 / 51) * 7 = (7 / 51) * 18 \approx 2$	2	8
2	9	27	$(27 / 51) * 7 = (7 / 51) * 27 \approx 4$	4	11
3	2	29	$(29 / 51) * 7 = (7 / 51) * 29 \approx 4$	4	
4	14	43	$(43 / 51) * 7 = (7 / 51) * 43 \approx 6$	6	15
5	1	44	$(44 / 51) * 7 = (7 / 51) * 44 \approx 6$	6	
6	5	49	$(49 / 51) * 7 = (7 / 51) * 49 \approx 7$	7	7
7	2	51	$(51 / 51) * 7 = (7 / 51) * 51 \approx 7$	7	

Intensity Transformation



■ Adjust brightness in gray images

- $J = \text{imadjust}(I, [\text{low_in}; \text{high_in}], [\text{low_out}; \text{high_out}])$

- Clipping values $< \text{low_in}$ and $> \text{high_in}$ TO
 - Values $< \text{low_in} \rightarrow \text{low_out}$, and values $> \text{high_in} \rightarrow \text{high_out}$

- Empty matrix ([]) specify the default of [0 1]

■ Examples– $J = \text{imadjust}(I, [0.3, 0.7], [0.2, 0.8]);$

■ Examples– $J = \text{imadjust}(I, [0.3, 0.7], []);$

